SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Class: III B.Tech - AI&DS **Semester : V**

Certified that this is the bonafide record of work done by **AJAY I** in the **20AD2E40 – FOUNDATIONS OF AGENTIC AI** of this institution for V semester during the academic year 2026-2027.

Faculty In-Charge	**Head Of The Department**
Mrs. C. Kavitha	**Dr. V. Karpagam**
AP(OG) / AI&DS	Professor & Head /AI&DS

Reg.No.: 71812401007

Submitted for the Autonomous Practical Examination held on _____

Internal Examiner **Subject Expert**

DEPARTMENT OF

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

VISION

To achieve excellence in the domain of Artificial Intelligence and Data Science and produce globally competent professionals providing multidisciplinary, sustainable solutions for societal challenges and industrial needs.

MISSION

- To deliver quality education to develop ethical and sustainable intelligent solutions for interdisciplinary domains.
- To promote research, innovation, and entrepreneurship through collaboration with industry and academia.
- To foster leadership and lifelong learning for professional and societal advancement.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

- **PEO 1:** Demonstrate professional and research competence using combined knowledge in Mathematics, Computing, Artificial Intelligence and Data Science
- **PEO 2:** Apply knowledge and skills in Artificial Intelligence and Data Science to develop innovative, sustainable and ethically responsible solutions to complex, multidisciplinary problems
- **PEO 3:** Engage in constructive research, professional growth and lifelong learning by effectively applying emerging technological expertise

PROGRAMME OUTCOMES (POs)

1. **Engineering knowledge**: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature and analyse complex engineering problems reaching substantiated conclusions with consideration for sustainable development (WK1 to WK4).

3. **Design/development of solutions**: Design creative solutions for complex engineering problems and design/ develop systems/ components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required (WK5).
4. **Conduct Investigations of Complex Problems**: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions (WK8).
5. **Engineering Tool Usage**: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems (WK2 and WK6).
6. **The Engineer and The World**: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment (WK1, WK5 and WK7).
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws (WK9).
8. **Individual and Collaborative Team work**: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication**: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective language, and learning differences.
10. **Project Management and Finance**: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member , leader in a team and to manage projects and in multidisciplinary environments.
11. **Life-Long Learning**: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change (WK8).

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO1: Analyze, design and develop sustainable intelligent solutions to address complex challenges posed by industry and society.

PSO2: Demonstrate data analysis skills to achieve effective insights and decision making to solve real-life problems.

PSO3: Apply mathematical and statistical techniques to model real-world problems using appropriate AI algorithms.

INDEX

EX. NO	DATE	NAME OF THE EXPERIMENT	PAGE NO
01.	01/08/2026	Implement the Observe–Orient–Decide–Act (OODA) loop for an AI agent	05
02.	08/08/2026	Develop an LLM-based agent using the Thought–Action cycle	07
03.	22/08/2026	Develop an LLM-based agent using the Observation cycle	09
04.	12/09/2026	Implement tool calling using LLM APIs	11
05.	19/09/2026	Implement structured output generation using LLM APIs	14
06.	02/10/2026	Compare System 1 and System 2 reasoning in autonomous agents	16
07.	03/10/2026	Develop autonomous agents using CoALA-based cognitive architecture	18
08.	14/10/2026	Design and implement Gymnasium-based intelligent agent environments	21
09.	15/10/2026	Implement agentic design patterns using ReAct, Reflection, and Planning	23
10.	16/10/2026	Develop a memory-enabled AI agent using RAG and MemGPT concepts	25
CONTENT BEYOND SYLLABUS			
	17/10/2026	Design and develop an intelligent agentic AI system integrating observation, reasoning, decision-making, tool usage, and memory for a real-world application	28

Ex No: 1	IMPLEMENT THE OBSERVE–ORIENT–DECIDE–ACT (OODA) LOOP FOR AN AI AGENT.
Date: 01/08/2026	

AIM:

To implement the Observe–Orient–Decide–Act (OODA) loop in Python for a study-assistant agent that observes the weather and free time, orients itself, decides and acts.

ALGORITHM:

1. Start the agent and repeat the OODA cycle for each situation.
2. Observe: read the temperature, rain condition and free hours from the user.
3. Orient: analyse the observations to decide whether outdoor activity and a long study session are possible.
4. Decide: select the most suitable activity from the analysed situation.
5. Act: execute (simulate) the selected action and display it.
6. Repeat the cycle for the next situation and stop after the last cycle.

PROGRAM:

```
def observe():
    print("--- OBSERVE ---")
    temperature = float(input("Enter temperature (C): "))
    raining = input("Is it raining? (yes/no): ").lower() == "yes"
    free_hours = float(input("Enter free hours available: "))
    return {"temperature": temperature, "raining": raining,
            "free_hours": free_hours}

def orient(obs):
    print("--- ORIENT ---")
    if obs["raining"]:
        outdoor = False
        print("Analysis: Rain makes outdoor activity unsuitable.")
    elif obs["temperature"] > 35:
        outdoor = False
        print("Analysis: The weather is too hot for outdoor activity.")
    else:
        outdoor = True
        print("Analysis: The weather is suitable for outdoor activity.")
    long_session = obs["free_hours"] >= 3
    print("Analysis: Long study session possible:", long_session)
    return {"outdoor": outdoor, "long_session": long_session}

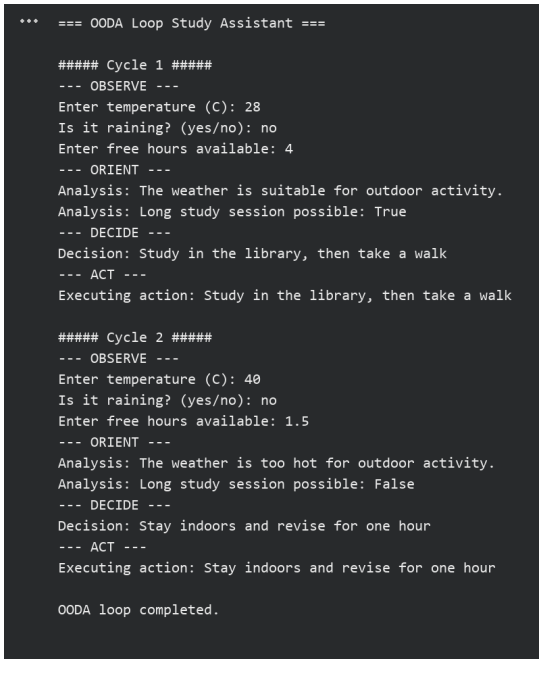
def decide(situation):
    print("--- DECIDE ---")
    if situation["outdoor"] and situation["long_session"]:
        decision = "Study in the library, then take a walk"
```

```
    elif situation["outdoor"]:
        decision = "Take a short walk and revise notes"
    elif situation["long_session"]:
        decision = "Stay indoors and complete a long study session"
    else:
        decision = "Stay indoors and revise for one hour"
    print("Decision:", decision)
    return decision

def act(decision):
    print("--- ACT ---")
    print("Executing action:", decision)

print("=== OODA Loop Study Assistant ===")
for cycle in range(1, 3):
    print(f"\n##### Cycle {cycle} #####")
    observation = observe()
    situation = orient(observation)
    decision = decide(situation)
    act(decision)
print("\nOODA loop completed.")
```

OUTPUT:



RESULT:

Thus, the experiment to implement the Observe–Orient–Decide–Act (OODA) loop for an AI agent was completed successfully.

Ex No: 2	DEVELOP AN LLM-BASED AGENT USING THE THOUGHT–ACTION CYCLE.
Date: 08/08/2026	

AIM:

To develop an agent that follows the Thought–Action cycle of LLM-based agents by interpreting a user task, selecting an action and executing it using Python.

Note: This is a rule-based simulation. The thoughts are produced by Python rules and not by a genuine large language model.

ALGORITHM:

1. Accept a task from the user.
2. Thought: interpret the task using keyword rules and numbers found in it.
3. Action: select the matching action (add, multiply, reverse or count words) with its arguments.
4. Execute the selected action using Python.
5. Display the thought, the action and the result.
6. Repeat until the user types exit.

PROGRAM:

```
import re

def think(task):
    text = task.lower()
    numbers = [float(n) for n in re.findall(r"\d+\.?\d*", text)]
    if "add" in text and len(numbers) >= 2:
        return "The task asks for addition.", "add", numbers
    if "multiply" in text and len(numbers) >= 2:
        return "The task asks for multiplication.", "multiply", numbers
    if "reverse" in text:
        word = text.split("reverse")[-1].strip()
        return "The task asks to reverse a word.", "reverse", [word]
    if "count" in text and "words" in text:
        sentence = task.split(":", 1)[-1].strip()
        return "The task asks to count words.", "count_words", [sentence]
    return "No matching action is available for this task.", "none", []

def act(action, args):
    if action == "add":
        return args[0] + args[1]
    if action == "multiply":
        return args[0] * args[1]
    if action == "reverse":
        return args[0][::-1]
    if action == "count_words":
```

```
        return len(args[0].split())
    return "Unable to complete the task"

print("=== Thought-Action Agent (Rule-Based Simulation) ===")
print("No LLM is used: thoughts are produced by Python rules.")
while True:
    task = input("\nEnter a task (or 'exit'): ").strip()
    if task.lower() == "exit":
        print("Agent stopped.")
        break
    thought, action, args = think(task)
    print("Thought:", thought)
    print("Action:", action, args)
    print("Result:", act(action, args))
```

OUTPUT:

```
=== Thought-Action Agent (Rule-Based Simulation) ===
No LLM is used: thoughts are produced by Python rules.

Enter a task (or 'exit'): Add 15 and 27
Thought: The task asks for addition.
Action: add [15.0, 27.0]
Result: 42.0

Enter a task (or 'exit'): Multiply 6 and 7
Thought: The task asks for multiplication.
Action: multiply [6.0, 7.0]
Result: 42.0

Enter a task (or 'exit'): Reverse agentic
Thought: The task asks to reverse a word.
Action: reverse ['agentic']
Result: citnega

Enter a task (or 'exit'): Count words: agents observe and act
Thought: The task asks to count words.
Action: count_words ['agents observe and act']
Result: 4

Enter a task (or 'exit'): Book a flight
Thought: No matching action is available for this task.
Action: none []
Result: Unable to complete the task

Enter a task (or 'exit'): exit
Agent stopped.
```

RESULT:

Thus, the experiment to develop an agent using the Thought–Action cycle was completed successfully as a rule-based simulation.

Ex No: 3	DEVELOP AN LLM-BASED AGENT USING THE OBSERVATION CYCLE.
Date: 22/08/2026	

AIM:

To develop an agent that repeatedly observes an environment, interprets each observation and selects an appropriate action, using a temperature-monitoring system.

ALGORITHM:

1. Initialise the sensor readings and the fan state as OFF.
2. Observe the temperature reading of the current cycle.
3. Interpret the reading as Cool, Warm or Hot.
4. Select an action for the fan based on the interpretation and the current fan state.
5. Update the fan state and display the action.
6. Repeat for all observation cycles and display the completion message.

PROGRAM:

```
readings = [24, 28, 33, 38, 31, 22]

def observe(cycle):
    value = readings[cycle]
    print(f"Cycle {cycle + 1}: Observation -> temperature = {value} C")
    return value

def interpret(value):
    if value < 26:
        return "Cool"
    if value < 32:
        return "Warm"
    return "Hot"

def select_action(state, fan):
    if state == "Hot":
        return "Turn fan ON at HIGH speed"
    if state == "Warm" and fan == "OFF":
        return "Turn fan ON at LOW speed"
    if state == "Cool" and fan != "OFF":
        return "Turn fan OFF"
    return "No change"

fan_state = "OFF"
print("=== Temperature Monitoring Agent ===")
for cycle in range(len(readings)):
    temperature = observe(cycle)
    state = interpret(temperature)
```

```
    print("         Interpretation ->", state)
    action = select_action(state, fan_state)
    print("         Action ->", action)
    if "HIGH" in action:
        fan_state = "HIGH"
    elif "LOW" in action:
        fan_state = "LOW"
    elif "OFF" in action:
        fan_state = "OFF"
    print("         Fan state ->", fan_state, "\n")
print("Monitoring completed after", len(readings), "observation cycles.")
```

OUTPUT:

```
=== Temperature Monitoring Agent ===
    Cycle 1: Observation -> temperature = 24 C
             Interpretation -> Cool
             Action -> No change
             Fan state -> OFF

    Cycle 2: Observation -> temperature = 28 C
             Interpretation -> Warm
             Action -> Turn fan ON at LOW speed
             Fan state -> LOW

    Cycle 3: Observation -> temperature = 33 C
             Interpretation -> Hot
             Action -> Turn fan ON at HIGH speed
             Fan state -> HIGH

    Cycle 4: Observation -> temperature = 38 C
             Interpretation -> Hot
             Action -> Turn fan ON at HIGH speed
             Fan state -> HIGH

    Cycle 5: Observation -> temperature = 31 C
             Interpretation -> Warm
             Action -> No change
             Fan state -> HIGH

    Cycle 6: Observation -> temperature = 22 C
             Interpretation -> Cool
             Action -> Turn fan OFF
             Fan state -> OFF

    Monitoring completed after 6 observation cycles.
```

RESULT:

Thus, the experiment to develop an agent using the observation cycle was completed successfully.

Ex No: 4	IMPLEMENT TOOL CALLING USING LLM APIS.
Date: 12/09/2026	

AIM:

To implement tool calling in an AI agent that selects a suitable tool for a user request, executes it and returns the result.

Note: This is a tool-calling simulation. The tool is selected by Python rules and no external LLM API is called. In a real implementation, the API key must be stored in an environment variable and never written in the source code.

ALGORITHM:

1. Define the tools: a calculator, a unit converter and a weather lookup.
2. Accept a request from the user.
3. Select the appropriate tool and extract its arguments from the request.
4. Execute the corresponding function with the arguments.
5. Display the selected tool, the arguments and the tool result.
6. Display a message when no tool matches, and stop when the user types exit.

PROGRAM:

```
import re

def calculator(a, b, op):
    if op == "+":
        return a + b
    if op == "-":
        return a - b
    if op == "*":
        return a * b
    return a / b if b != 0 else "Division by zero"

def unit_converter(value, unit):
    factors = {"km": ("miles", 0.621371), "kg": ("pounds", 2.20462)}
    if unit == "c":
        return f"{value * 9 / 5 + 32:.1f} F"
    name, factor = factors[unit]
    return f"{value * factor:.2f} {name}"

def weather(city):
    data = {"chennai": "32 C, humid", "coimbatore": "27 C, cloudy"}
    return data.get(city, "No data available")

def select_tool(request):
    text = request.lower()
    nums = [float(n) for n in re.findall(r"\d+\.?\d*", text)]
```

```
    if "weather" in text:
        city = text.split("in")[-1].strip()
        return "weather", (city,)
    for unit in ("km", "kg"):
        if unit in text and nums:
            return "unit_converter", (nums[0], unit)
    if "celsius" in text and nums:
        return "unit_converter", (nums[0], "c")
    match = re.search(r"(\d+\.?\d*)\s*([+\-*/])\s*(\d+\.?\d*)", text)
    if match:
        a, op, b = match.groups()
        return "calculator", (float(a), float(b), op)
    return None, ()

tools = {"calculator": calculator, "unit_converter": unit_converter,
         "weather": weather}
print("=== Tool Calling Agent (Simulation, no external LLM API) ===")
while True:
    request = input("\nUser request (or 'exit'): ").strip()
    if request.lower() == "exit":
        print("Agent stopped.")
        break
    name, args = select_tool(request)
    if name is None:
        print("Agent: No suitable tool found.")
        continue
    print("Tool selected :", name)
    print("Arguments     :", args)
    print("Tool result   :", tools[name](*args))
```

OUTPUT:

```
***  === Tool Calling Agent (Simulation, no external LLM API) ===

     User request (or 'exit'): What is 12 * 8
     Tool selected : calculator
     Arguments     : (12.0, 8.0, '*')
     Tool result   : 96.0

     User request (or 'exit'): Convert 10 km
     Tool selected : unit_converter
     Arguments     : (10.0, 'km')
     Tool result   : 6.21 miles

     User request (or 'exit'): Convert 36 celsius
     Tool selected : unit_converter
     Arguments     : (36.0, 'c')
     Tool result   : 96.8 F

     User request (or 'exit'): What is the weather in Coimbatore
     Tool selected : weather
     Arguments     : ('coimbatore',)
     Tool result   : 27 C, cloudy

     User request (or 'exit'): Sing a song
     Agent: No suitable tool found.

     User request (or 'exit'): exit
     Agent stopped.
```

RESULT:

Thus, the experiment to implement tool calling was completed successfully as a tool-calling simulation.

Ex No: 5	IMPLEMENT STRUCTURED OUTPUT GENERATION USING LLM APIS.
Date: 19/09/2026	

AIM:

To generate structured output in JSON format from a study request and to validate the generated fields.

Note: This is a structured-output simulation. The fields are produced by Python rules and no LLM API is called.

ALGORITHM:

1. Accept a study request from the user.
2. Extract the task name, priority and duration to form a Python dictionary.
3. Add the default status field as pending.
4. Validate the task, the priority and the duration.
5. Display the valid JSON output using the json module.
6. Display the validation errors if the output is invalid.

PROGRAM:

```
import json
import re

PRIORITIES = ("low", "medium", "high")

def generate(request):
    text = request.lower()
    priority = "medium"
    for level in PRIORITIES:
        if level in text:
            priority = level
    match = re.search(r"(\d+\.?\d*)\s*hour", text)
    duration = float(match.group(1)) if match else None
    task = re.split(r",| for \d", request)[0].strip()
    return {"task": task, "priority": priority,
            "duration_hours": duration, "status": "pending"}

def validate(data):
    errors = []
    if not data["task"]:
        errors.append("task must not be empty")
    if data["priority"] not in PRIORITIES:
        errors.append("priority must be low, medium or high")
    if not isinstance(data["duration_hours"], float) or \
            data["duration_hours"] <= 0:
        errors.append("duration_hours must be a positive number")
```

```
    return errors

print("=== Structured Output Generator (Simulation) ===")
while True:
    request = input("\nEnter a study request (or 'exit'): ").strip()
    if request.lower() == "exit":
        print("Generator stopped.")
        break
    output = generate(request)
    problems = validate(output)
    if problems:
        print("Validation FAILED:", "; ".join(problems))
    else:
        print("Validation PASSED. Valid JSON output:")
        print(json.dumps(output, indent=2))
```

OUTPUT:

```
***  === Structured Output Generator (Simulation) ===

     Enter a study request (or 'exit'): Prepare Agentic AI record, high priority, for 2 hours
     Validation PASSED. Valid JSON output:
     {
       "task": "Prepare Agentic AI record",
       "priority": "high",
       "duration_hours": 2.0,
       "status": "pending"
     }

     Enter a study request (or 'exit'): Revise notes with low priority
     Validation FAILED: duration_hours must be a positive number

     Enter a study request (or 'exit'): exit
     Generator stopped.
```

RESULT:

Thus, the experiment to generate and validate structured JSON output was completed successfully as a simulation.

Ex No: 6	COMPARE SYSTEM 1 AND SYSTEM 2 REASONING IN AUTONOMOUS AGENTS.
Date: 02/10/2026	

AIM:

To compare fast, rule-based System 1 decisions with slower, multi-criteria System 2 analysis for selecting a laptop within a budget.

ALGORITHM:

1. Store the price, speed and battery details of the laptops.
2. System 1: apply a single quick rule and choose the cheapest laptop within the budget.
3. System 2: calculate a weighted score using price, speed and battery for every affordable laptop.
4. Choose the laptop with the highest score and count the steps used by each system.
5. Compare the two decisions and display whether they are the same.
6. Repeat for different budgets.

PROGRAM:

```
laptops = [
    {"name": "Laptop A", "price": 38000, "speed": 5, "battery": 6},
    {"name": "Laptop B", "price": 55000, "speed": 8, "battery": 9},
    {"name": "Laptop C", "price": 72000, "speed": 9, "battery": 7},
    {"name": "Laptop D", "price": 45000, "speed": 7, "battery": 5},
]

def system1(budget):
    """Fast rule: choose the cheapest laptop within budget."""
    steps = 0
    best = None
    for item in laptops:
        steps += 1
        if item["price"] <= budget:
            if best is None or item["price"] < best["price"]:
                best = item
    return best, steps

def system2(budget):
    """Slow analysis: weighted score over price, speed and battery."""
    steps = 0
    best, best_score = None, -1
    for item in laptops:
        if item["price"] > budget:
            steps += 1
            continue
        price_score = (budget - item["price"]) / budget * 10
```

```
        score = (0.3 * price_score + 0.4 * item["speed"]
                + 0.3 * item["battery"])
        steps += 4
        if score > best_score:
            best, best_score = item, score
    return best, best_score, steps

print("=== System 1 vs System 2 Reasoning ===")
for budget in (40000, 60000, 80000):
    print(f"\nBudget: Rs. {budget}")
    fast, steps1 = system1(budget)
    slow, score, steps2 = system2(budget)
    print(f"System 1 (fast rule)  : {fast['name']} (steps = {steps1})")
    print(f"System 2 (analysis)   : {slow['name']} "
        f"(score = {score:.2f}, steps = {steps2})")
    same = fast["name"] == slow["name"]
    print("Comparison            :", "Same decision" if same
        else "Different decision - deeper analysis changed the choice")
```

OUTPUT:

```
***  === System 1 vs System 2 Reasoning ===

     Budget: Rs. 40000
     System 1 (fast rule)  : Laptop A (steps = 4)
     System 2 (analysis)   : Laptop A (score = 3.95, steps = 7)
     Comparison            : Same decision

     Budget: Rs. 60000
     System 1 (fast rule)  : Laptop A (steps = 4)
     System 2 (analysis)   : Laptop B (score = 6.15, steps = 13)
     Comparison            : Different decision - deeper analysis changed the choice

     Budget: Rs. 80000
     System 1 (fast rule)  : Laptop A (steps = 4)
     System 2 (analysis)   : Laptop B (score = 6.84, steps = 16)
     Comparison            : Different decision - deeper analysis changed the choice
```

RESULT:

Thus, the experiment to compare System 1 and System 2 reasoning was completed successfully. System 2 used more steps but selected better laptops when the budget increased.

Ex No: 7	DEVELOP AUTONOMOUS AGENTS USING COALA-BASED COGNITIVE ARCHITECTURE.
Date: 03/10/2026	

AIM:

To develop an educational agent inspired by the CoALA cognitive architecture using working memory, episodic memory, semantic memory and action selection.

Note: This is a simplified educational agent inspired by CoALA and is not the official framework.

ALGORITHM:

1. Create an environment with a temperature and an air conditioner state.
2. Create the agent with working memory, episodic memory and semantic memory.
3. Observe the environment and store the observation in working memory.
4. Recall similar past episodes and consult the semantic knowledge.
5. Select an action, execute it in the environment and store the episode.
6. Repeat for several steps and display the episodic memory.

PROGRAM:

```
class Environment:
    def __init__(self):
        self.temperature = 28
        self.ac_on = False

    def observe(self):
        return {"temperature": self.temperature, "ac_on": self.ac_on}

    def execute(self, action):
        if action == "turn_on_ac":
            self.ac_on = True
        elif action == "turn_off_ac":
            self.ac_on = False
        self.temperature += -2 if self.ac_on else 3
        return self.temperature

class CoALAAgent:
    def __init__(self):
        self.working = {}
        self.episodic = []
        self.semantic = {"hot": "temperature above 30 needs cooling",
                         "cool": "temperature below 25 needs no cooling"}

    def decide(self, obs):
        self.working = dict(obs)
        temp = obs["temperature"]
```

```
        past = [e for e in self.episodic if e["temperature"] == temp]
        if past:
            last = past[-1]
            print(f"  Episodic recall : at {temp} C, '{last['action']}' "
                  f"gave {last['result']} C")
        if temp > 30 and not obs["ac_on"]:
            print("  Semantic memory :", self.semantic["hot"])
            return "turn_on_ac"
        if temp < 25 and obs["ac_on"]:
            print("  Semantic memory :", self.semantic["cool"])
            return "turn_off_ac"
        return "wait"

    def remember(self, action, new_temp):
        self.episodic.append({"temperature": self.working["temperature"],
                              "action": action, "result": new_temp})

env, agent = Environment(), CoALAAgent()
print("=== CoALA-Inspired Cognitive Agent ===")
for step in range(1, 10):
    obs = env.observe()
    print(f"Step {step}: working memory = {obs}")
    action = agent.decide(obs)
    result = env.execute(action)
    agent.remember(action, result)
    print(f"  Action selected : {action} -> new temperature {result}")
print("\nEpisodic memory size:", len(agent.episodic), "episodes")
print("Last episode:", agent.episodic[-1])
```

OUTPUT:

```
=== CoALA-Inspired Cognitive Agent ===
Step 1: working memory = {'temperature': 28, 'ac_on': False}
  Action selected : wait -> new temperature 31
Step 2: working memory = {'temperature': 31, 'ac_on': False}
  Semantic memory : temperature above 30 needs cooling
  Action selected : turn_on_ac -> new temperature 29
Step 3: working memory = {'temperature': 29, 'ac_on': True}
  Action selected : wait -> new temperature 27
Step 4: working memory = {'temperature': 27, 'ac_on': True}
  Action selected : wait -> new temperature 25
Step 5: working memory = {'temperature': 25, 'ac_on': True}
  Action selected : wait -> new temperature 23
Step 6: working memory = {'temperature': 23, 'ac_on': True}
  Semantic memory : temperature below 25 needs no cooling
  Action selected : turn_off_ac -> new temperature 26
Step 7: working memory = {'temperature': 26, 'ac_on': False}
  Action selected : wait -> new temperature 29
Step 8: working memory = {'temperature': 29, 'ac_on': False}
  Episodic recall : at 29 C, 'wait' gave 27 C
  Action selected : wait -> new temperature 32
Step 9: working memory = {'temperature': 32, 'ac_on': False}
  Semantic memory : temperature above 30 needs cooling
  Action selected : turn_on_ac -> new temperature 30

Episodic memory size: 9 episodes
Last episode: {'temperature': 32, 'action': 'turn_on_ac', 'result': 30}
```

RESULT:

Thus, the experiment to develop an agent using a CoALA-inspired cognitive architecture was completed successfully.

Ex No: 8	DESIGN AND IMPLEMENT GYMNASIUM-BASED INTELLIGENT AGENT ENVIRONMENTS.
Date: 14/10/2026	

AIM:

To design a custom grid-world environment using Gymnasium and to demonstrate observation spaces, action spaces, reset, step, rewards and termination.

ALGORITHM:

1. Install Gymnasium and import the required modules.
2. Create a custom environment class with a discrete observation space and a discrete action space.
3. Implement reset() to return the start state.
4. Implement step() to return state, reward, terminated and truncated.
5. Execute a sequence of actions in the environment and display each step.
6. Display the total reward when the goal is reached.

Install necessary libraries:

pip install gymnasium

PROGRAM:

```
import gymnasium as gym
from gymnasium import spaces

class GridWorldEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.observation_space = spaces.Discrete(16)
        # Actions: 0=Up 1=Down 2=Left 3=Right
        self.action_space = spaces.Discrete(4)
        self.state = 0
        self.steps = 0

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.state, self.steps = 0, 0
        return self.state, {}

    def step(self, action):
        row, col = divmod(self.state, 4)
        if action == 0:
            row = max(row - 1, 0)
        elif action == 1:
            row = min(row + 1, 3)
        elif action == 2:
            col = max(col - 1, 0)
```

```
        else:
            col = min(col + 1, 3)
        self.state = row * 4 + col
        self.steps += 1
        terminated = self.state in (5, 15)
        reward = -0.1
        if self.state == 15:
            reward = 10.0
        elif self.state == 5:
            reward = -5.0
        truncated = self.steps >= 20
        return self.state, reward, terminated, truncated, {}

env = GridWorldEnv()
print("Observation space:", env.observation_space)
print("Action space     :", env.action_space)
obs, info = env.reset(seed=42)
print("Reset -> start state:", obs)
policy = [3, 3, 3, 1, 1, 1]  # Right x3, Down x3
names = ["Up", "Down", "Left", "Right"]
total = 0.0
for action in policy:
    obs, reward, terminated, truncated, info = env.step(action)
    total += reward
    print(f"Action {names[action]:5s} -> state {obs:2d}, "
          f"reward {reward:5.1f}, terminated={terminated}")
    if terminated or truncated:
        break
print("Total reward:", round(total, 1))
```

OUTPUT:

```
***  Observation space: Discrete(16)
     Action space     : Discrete(4)
     Reset -> start state: 0
     Action Right -> state  1, reward  -0.1, terminated=False
     Action Right -> state  2, reward  -0.1, terminated=False
     Action Right -> state  3, reward  -0.1, terminated=False
     Action Down  -> state  7, reward  -0.1, terminated=False
     Action Down  -> state 11, reward  -0.1, terminated=False
     Action Down  -> state 15, reward  10.0, terminated=True
     Total reward: 9.5
```

RESULT:

Thus, the experiment to design and implement a Gymnasium-based grid-world environment was completed successfully.

Ex No: 9	IMPLEMENT AGENTIC DESIGN PATTERNS USING REACT, REFLECTION, AND PLANNING.
Date: 15/10/2026	

AIM:

To implement the ReAct, Reflection and Planning design patterns in an automated study-planner agent.

ALGORITHM:

1. Accept the available study hours from the user.
2. Planning: divide the exam goal into smaller tasks and adjust the plan to fit the available hours.
3. ReAct: for each task, display the thought, perform the action and record the observation.
4. Reflection: evaluate the observed score and decide whether an improvement is needed.
5. Add a revision slot for any task with a low score.
6. Display the reflection summary and the final plan.

PROGRAM:

```
scores = {"Revise notes": 85, "Practice programs": 55, "Mock test": 70}

def plan(goal, hours):
    print("PLANNING: Goal ->", goal)
    tasks = [("Revise notes", 2), ("Practice programs", 3),
             ("Mock test", 2)]
    total = sum(h for _, h in tasks)
    print("Planned tasks:", tasks, "| total", total, "h")
    if total > hours:
        tasks = [(n, max(1, h - 1)) for n, h in tasks]
        print("Plan adjusted to fit", hours, "h:", tasks)
    return tasks

def react(task, hours):
    print(f"\nReAct -> Task: {task}")
    print(f"  Thought    : I need {hours} h to complete '{task}'.")
    print(f"  Action     : Start '{task}' for {hours} h.")
    result = scores[task]
    print(f"  Observation: Self-test score = {result}%")
    return result

def reflect(task, result):
    if result < 60:
        print(f"  Reflection : Score {result}% is low; add revision.")
        return True
```

```
    print(f"  Reflection : Score {result}% is acceptable.")
    return False

print("=== ReAct, Reflection and Planning Study Agent ===")
available = int(input("Enter available hours: "))
tasks = plan("Prepare for the Agentic AI exam", available)
extra = []
for name, hours in tasks:
    result = react(name, hours)
    if reflect(name, result):
        extra.append(name)
print("\nREFLECTION SUMMARY")
if extra:
    print("Revision slots added for:", ", ".join(extra))
    print("Final plan updated: 1 h revision of", extra[0])
else:
    print("No changes required.")
print("Study plan completed.")
```

OUTPUT:

```
***  === ReAct, Reflection and Planning Study Agent ===
     Enter available hours: 6
     PLANNING: Goal -> Prepare for the Agentic AI exam
     Planned tasks: [('Revise notes', 2), ('Practice programs', 3), ('Mock test', 2)] | total 7 h
     Plan adjusted to fit 6 h: [('Revise notes', 1), ('Practice programs', 2), ('Mock test', 1)]

     ReAct -> Task: Revise notes
       Thought    : I need 1 h to complete 'Revise notes'.
       Action     : Start 'Revise notes' for 1 h.
       Observation: Self-test score = 85%
       Reflection : Score 85% is acceptable.

     ReAct -> Task: Practice programs
       Thought    : I need 2 h to complete 'Practice programs'.
       Action     : Start 'Practice programs' for 2 h.
       Observation: Self-test score = 55%
       Reflection : Score 55% is low; add revision.

     ReAct -> Task: Mock test
       Thought    : I need 1 h to complete 'Mock test'.
       Action     : Start 'Mock test' for 1 h.
       Observation: Self-test score = 70%
       Reflection : Score 70% is acceptable.

     REFLECTION SUMMARY
     Revision slots added for: Practice programs
     Final plan updated: 1 h revision of Practice programs
     Study plan completed.
```

RESULT:

Thus, the experiment to implement ReAct, Reflection and Planning patterns was completed successfully.

Ex No: 10	DEVELOP A MEMORY-ENABLED AI AGENT USING RAG AND MEMGPT CONCEPTS.
Date: 16/10/2026	

AIM:

To develop a memory-enabled agent that retrieves information from a small local knowledge base using RAG concepts and keeps user preferences in SQLite across sessions.

Note: This is a simplified educational implementation. It is not the official MemGPT system or a production RAG pipeline.

ALGORITHM:

1. Create a small knowledge base and an SQLite table for persistent memory.
2. Load the stored preferences when the program starts.
3. Store a preference when the user enters a remember command.
4. Retrieve the most relevant document for a question using keyword matching.
5. Generate the answer from the retrieved document and apply the stored answer style.
6. Restart the program and verify that the preferences are retrieved from the database.

PROGRAM:

```
import re
import sqlite3

knowledge_base = [
    ("RAG", "Retrieval-Augmented Generation retrieves documents and uses "
            "them to answer. It reduces made-up answers."),
    ("MemGPT", "MemGPT-style agents move facts between core memory and "
               "archival memory. This gives the agent long-term memory."),
    ("Tool calling", "Tool calling lets an agent select a function and "
                     "pass arguments to it. The result is used in reply."),
]

db = sqlite3.connect("rag_memory.db")
db.execute("CREATE TABLE IF NOT EXISTS memory "
           "(key TEXT PRIMARY KEY, value TEXT)")

def remember(key, value):
    db.execute("INSERT OR REPLACE INTO memory VALUES (?, ?)",
               (key, value))
    db.commit()

def recall(key):
    row = db.execute("SELECT value FROM memory WHERE key = ?",
                     (key,)).fetchone()
    return row[0] if row else None
```

```
STOP = {"what", "is", "the", "a", "an", "of", "to", "and", "it", "in"}

def tokens(text):
    return set(re.findall(r"\w+", text.lower())) - STOP

def retrieve(question):
    words = tokens(question)
    best, best_score = None, 0
    for title, text in knowledge_base:
        score = len(words & tokens(title + " " + text))
        if score > best_score:
            best, best_score = (title, text), score
    return best

print("=== RAG Agent with Persistent Memory ===")
print("Stored preferences:", recall("style"), "|", recall("topic"))
while True:
    line = input("\nYou: ").strip()
    if line.lower() == "exit":
        print("Session ended. Memory saved to rag_memory.db")
        break
    if line.lower().startswith("remember "):
        key, value = line[9:].split("=")
        remember(key.strip(), value.strip())
        print("Agent: Saved preference", key.strip(), "=", value.strip())
        continue
    if line.lower().startswith("what is my"):
        key = line.lower().split("my")[-1].strip(" ?")
        print("Agent: Your", key, "is", recall(key))
        continue
    doc = retrieve(line)
    if doc is None:
        print("Agent: No relevant document found.")
        continue
    answer = doc[1]
    if recall("style") == "short":
        answer = answer.split(".")[0] + "."
    print("Agent: [Retrieved:", doc[0] + "]", answer)
```

OUTPUT:

Session 1:

```
***  === RAG Agent with Persistent Memory ===
     Stored preferences: None | None

     You: remember style = short
     Agent: Saved preference style = short

     You: remember topic = RAG
     Agent: Saved preference topic = RAG

     You: What is RAG?
     Agent: [Retrieved: RAG] Retrieval-Augmented Generation retrieves documents and uses them to answer.

     You: exit
     Session ended. Memory saved to rag_memory.db
```

Session 2 (after restarting the program):

```
***  === RAG Agent with Persistent Memory ===
     Stored preferences: short | RAG

     You: What is my style?
     Agent: Your style is short

     You: What is my topic?
     Agent: Your topic is RAG

     You: Explain MemGPT memory
     Agent: [Retrieved: MemGPT] MemGPT-style agents move facts between core memory and archival memory.

     You: What is tool calling?
     Agent: [Retrieved: Tool calling] Tool calling lets an agent select a function and pass arguments to it.

     You: exit
     Session ended. Memory saved to rag_memory.db
```

RESULT:

Thus, the experiment to develop a memory-enabled agent using RAG and MemGPT concepts was completed successfully, and the preferences stored in Session 1 were retrieved in Session 2.

Ex No: CBS	DESIGN AND DEVELOP AN INTELLIGENT AGENTIC AI SYSTEM INTEGRATING OBSERVATION, REASONING, DECISION-MAKING, TOOL USAGE, AND MEMORY FOR A REAL-WORLD APPLICATION.
Date: 17/10/2026	

AIM:

To design and develop an intelligent study assistant that integrates observation, reasoning, decision-making, tool usage and memory.

ALGORITHM:

1. Open the SQLite database and load the stored preferences from memory.
2. Observe the subject, days left, available hours and energy level from the user.
3. Reason about the exam urgency and the past rating of the subject stored in memory.
4. Decide the study mode: revision sprint, practice session or light reading.
5. Use the tools to split the study time and to fetch a study tip.
6. Store the session rating and the preference in memory for future decisions.
7. Run the assistant again and show that the stored memory changes the decision.

PROGRAM:

```
import sqlite3

db = sqlite3.connect("study_assistant.db")
db.execute("CREATE TABLE IF NOT EXISTS sessions "
        "(subject TEXT, hours REAL, rating INTEGER)")
db.execute("CREATE TABLE IF NOT EXISTS preferences "
        "(key TEXT PRIMARY KEY, value TEXT)")

def split_time(hours, parts):
    """Tool 1: divide the study time into equal blocks."""
    return round(hours / parts, 2)

def get_tip(subject):
    """Tool 2: look up a study tip."""
    tips = {"agentic ai": "Build a small agent after each topic.",
         "maths": "Solve five problems daily."}
    return tips.get(subject.lower(), "Make short notes and revise daily.")

def past_rating(subject):
    sql = "SELECT AVG(rating) FROM sessions WHERE subject = ?"
    return db.execute(sql, (subject,)).fetchone()[0]

print("=== Intelligent Study Assistant ===")
row = db.execute("SELECT value FROM preferences WHERE key = 'style'")
```

```
row = row.fetchone()
style = row[0] if row else "mixed"
print("Memory -> preferred style:", style)

subject = input("Subject: ")
days = int(input("Days left for exam: "))
hours = float(input("Hours available today: "))
energy = input("Energy level (low/medium/high): ").lower()
print(f"\nOBSERVE: {subject} | exam in {days} days | {hours} h | {energy}")

rating = past_rating(subject)
weak = rating is not None and rating < 3
print("REASON : exam is", "near" if days <= 7 else "far",
      "| past rating:", "none" if rating is None else round(rating, 1))

if days <= 3:
    decision, parts = "Revision sprint", 3
elif weak or energy == "high":
    decision, parts = "Practice session", 2
else:
    decision, parts = "Light reading", 2
print("DECIDE :", decision)

block = split_time(hours, parts)
print("TOOL   : split_time ->", parts, "blocks of", block, "h")
print("TOOL   : get_tip    ->", get_tip(subject))

score = int(input("\nRate this session (1-5): "))
db.execute("INSERT INTO sessions VALUES (?, ?, ?)",
           (subject, hours, score))
db.execute("INSERT OR REPLACE INTO preferences VALUES ('style', ?)",
           ("practice" if decision == "Practice session" else style,))
db.commit()
print("MEMORY : session saved for future decisions.")
```

OUTPUT:

Run 1 (no stored experience):

```
***  === Intelligent Study Assistant ===
     Memory -> preferred style: mixed
     Subject: Agentic AI
     Days left for exam: 10
     Hours available today: 4
     Energy level (low/medium/high): high

     OBSERVE: Agentic AI | exam in 10 days | 4.0 h | high
     REASON : exam is far | past rating: none
     DECIDE : Practice session
     TOOL   : split_time -> 2 blocks of 2.0 h
     TOOL   : get_tip    -> Build a small agent after each topic.

     Rate this session (1-5): 2
     MEMORY : session saved for future decisions.
```

Run 2 (after storing the first session):

```
***  === Intelligent Study Assistant ===
     Memory -> preferred style: practice
     Subject: Agentic AI
     Days left for exam: 5
     Hours available today: 3
     Energy level (low/medium/high): low

     OBSERVE: Agentic AI | exam in 5 days | 3.0 h | low
     REASON : exam is near | past rating: 2.0
     DECIDE : Practice session
     TOOL   : split_time -> 2 blocks of 1.5 h
     TOOL   : get_tip    -> Build a small agent after each topic.

     Rate this session (1-5): 4
     MEMORY : session saved for future decisions.
```

RESULT:

Thus, the intelligent agentic AI study assistant integrating observation, reasoning, decision-making, tool usage and memory was designed, developed and executed successfully.